Name : Janhvi Ramrakhyani **Semester :** V **Branch :** AI&DS

Roll No : 89 **Batch :** T21

Experiment No. 08

Aim : To implement CI/CD pipeline to host a website on AWS

LO Mapped : LO3

LO : Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment.

1. Aim / Objective

To design, implement, and test an automated End-to-End Continuous Integration and Continuous Deployment (CI/CD) pipeline on Amazon Web Services (AWS) using GitHub, AWS CodePipeline, AWS CodeBuild, and Amazon S3 for hosting a dynamic/static web application.

2. Theoretical Background

What is Amazon Web Services (AWS)?

Amazon Web Services (AWS) is a comprehensive, evolving cloud computing platform provided by Amazon that offers a mixture of Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) offerings. AWS provides on-demand delivery of compute power, database storage, application deployment, and other IT resources with a pay-as-you-go pricing model.

What is a CI/CD Pipeline?

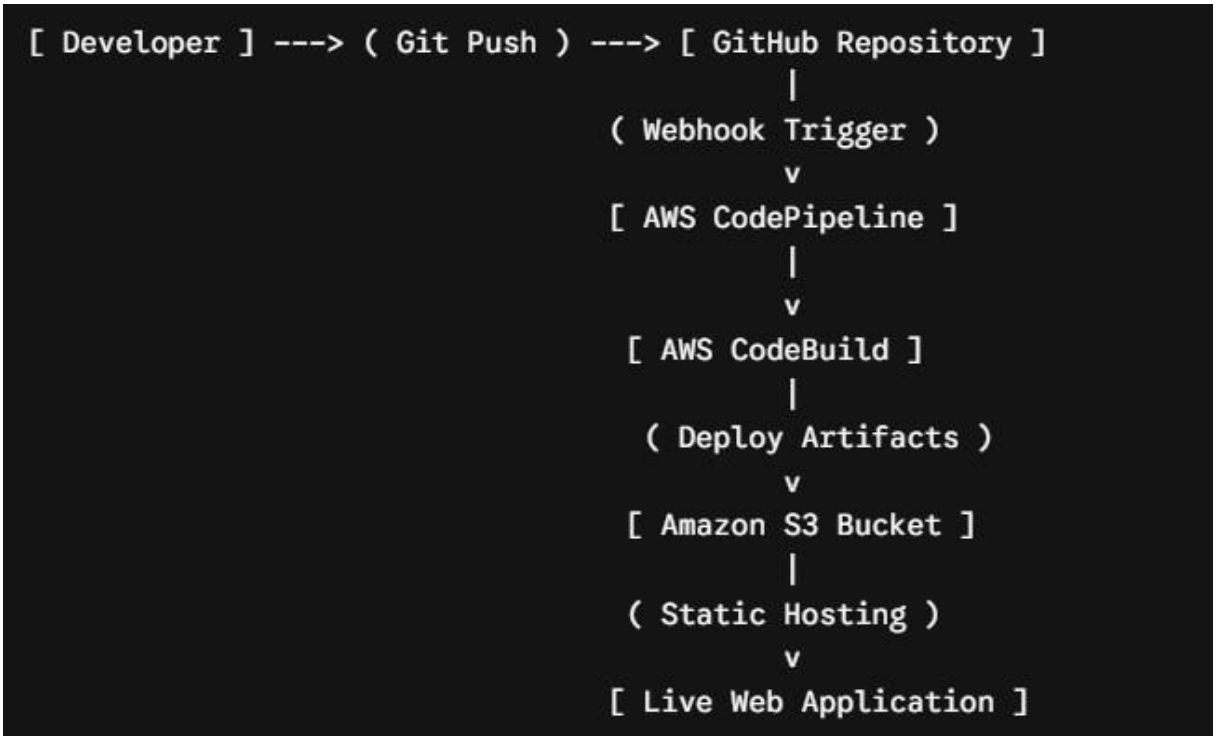
- **Continuous Integration (CI):** A software development practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run. The primary goal is to find and address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.
- **Continuous Deployment (CD):** An extension of continuous integration, where code changes are automatically built, tested, and prepared for release to production. It automates the entire software release process up to deployment to the live

environment.

AWS Services Utilized in the CI/CD Pipeline Architecture

1. **GitHub (Source Provider):** Hosts the application source code repository. Webhooks trigger the AWS pipeline automatically upon every push/commit to the target branch.
2. **AWS CodePipeline:** An enterprise orchestrator service that models, visualizes, and automates the steps required to release software. It monitors source code repositories and triggers build and deployment stages upon detecting code revisions.
3. **AWS CodeBuild:** A fully managed containerized build service that compiles source code, runs unit/integration tests, and produces deployable software artifacts based on instructions defined in a buildspec.yml file.
4. **Amazon S3 (Simple Storage Service):** An object storage service used for dual purposes in this pipeline: storing pipeline build artifacts securely and serving as a high-availability hosting platform via S3 Static Website Hosting.
5. **AWS IAM (Identity and Access Management):** Controls authentication and authorization by granting precise service roles and permissions (e.g., granting CodeBuild rights to access S3 resources).

3. Architecture Diagram & Workflow



4. Step-by-Step Implementation Procedure

Phase 1: AWS CodeBuild Project Configuration

Step 1: Create a Build Project

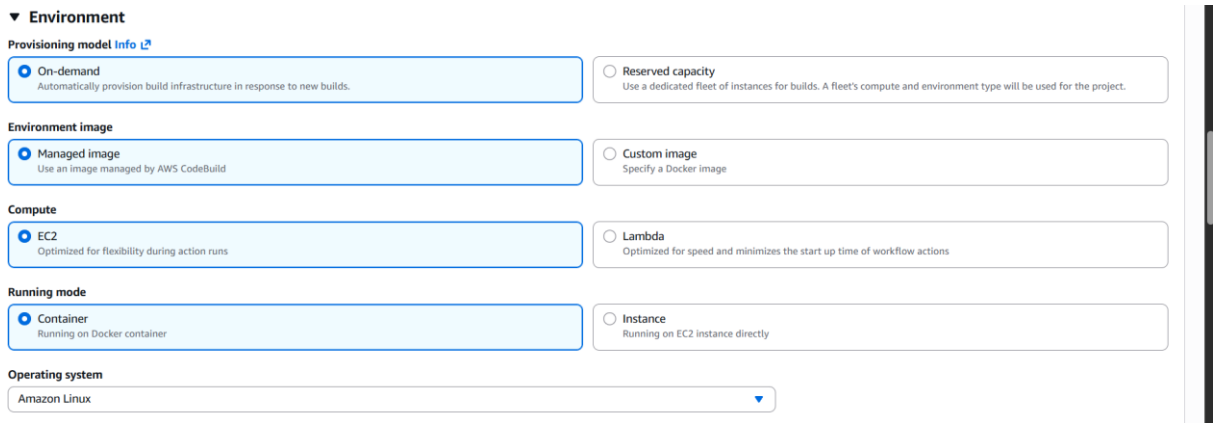
1. Open the AWS Management Console and navigate to **Developer Tools > CodeBuild**.
2. Click on **Create build project**.
3. Under **Project configuration**, set the **Project name** to pwa-codebuild-vb.
4. Select **Default project** as the project type.

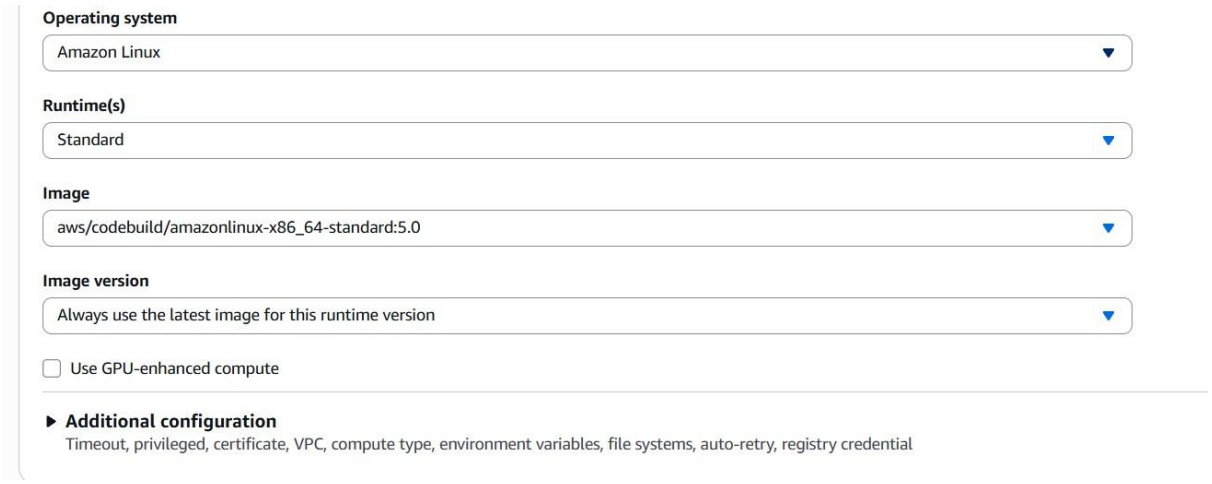
Step 2: Configure Source & Authenticate GitHub

1. Under the **Source** section, select **GitHub** as the Source provider.
2. Select **OAuth app** as the credential type.
3. Click on **Connect to GitHub** and complete the OAuth authorization prompt (**Authorize aws-codesuite**) to link your GitHub account.

Step 3: Repository Selection & Environment Setup

1. Choose **Repository in my GitHub account**.
2. Select the repository URL (e.g., https://github.com/vishal003/Car-pwa-deploy-on-aws.git).
3. Scroll down to **Environment**:
    - **Provisioning model:** On-demand
    - **Environment image:** Managed image
    - **Compute:** EC2
    - **Running mode:** Instance
    - **Operating system:** Amazon Linux
    - **Runtime(s):** Base
4. Under **Service role**, choose **New service role** and verify the generated role name (codebuild-pwa-codebuild-vb-service-role).
5. Under **Buildspec**, select **Use a buildspec file** (defaulting to buildspec.yml in the repository root).
6. Under **Artifacts**, select **No artifacts** (as CodePipeline will manage artifacts). Enable CloudWatch logs and click **Create build project**.

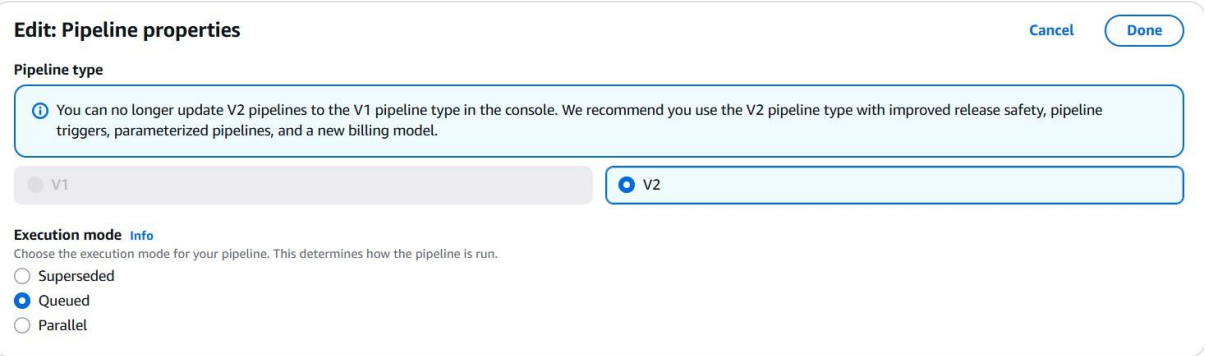




Phase 2: AWS CodePipeline Setup

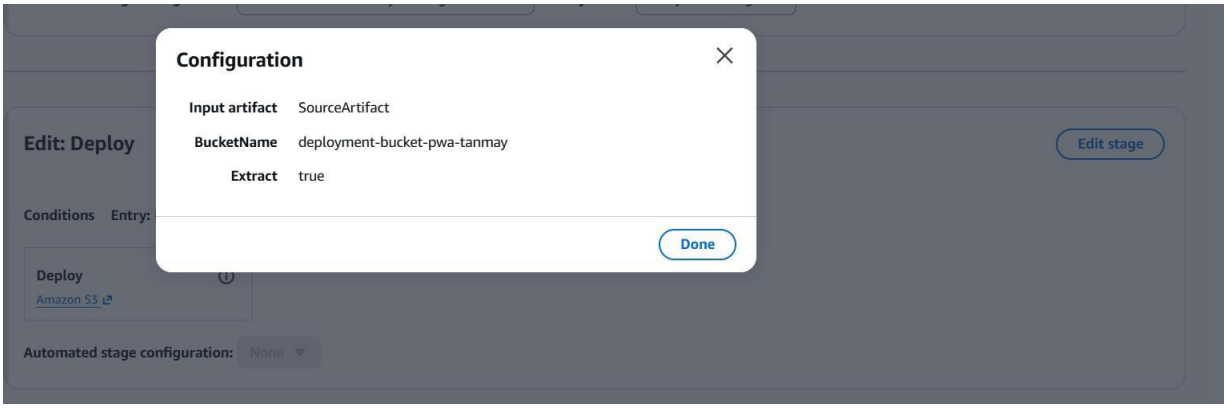
Step 4: Pipeline Settings & Artifact Storage

1. Navigate to **Developer Tools > CodePipeline** and click **Create pipeline**.
2. Enter **Pipeline name** as pwa-pipeline.
3. Set **Execution mode** to **Queued** and select **New service role**.
4. Expand **Advanced settings**:
    - **Artifact store:** Custom location
    - **Bucket:** Select your S3 artifact bucket (e.g., store-build-artifact-vb)
    - **Encryption key:** Default AWS Managed Key
5. Click **Next**.



Step 5: Add Source, Build, and Deploy Stages

1. **Source Stage:**
    - **Source provider:** GitHub (via OAuth app)
    - Select Repository (vishal003/Car-pwa-deploy-on-aws) and Branch (master).
2. **Build Stage:**
    - **Build provider:** AWS CodeBuild
    - **Project name:** Select pwa-codebuild-vb
    - **Build type:** Single build
3. **Deploy Stage:**
    - **Deploy provider:** Amazon S3
    - **Bucket:** Select deployment bucket (deployment-bucket-pwa-vb)
    - Check **Extract file before deploy**.
4. Review all settings and click **Create pipeline**.

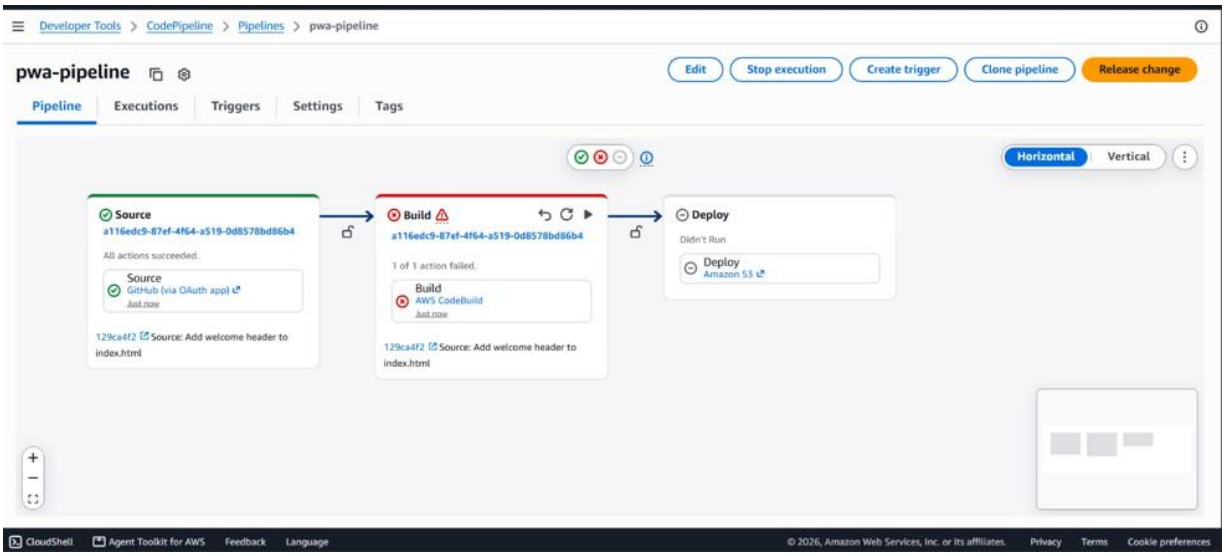


Phase 3: Troubleshooting IAM Permissions & Fixing Pipeline Build Failure

Step 6: Identify Build Failure

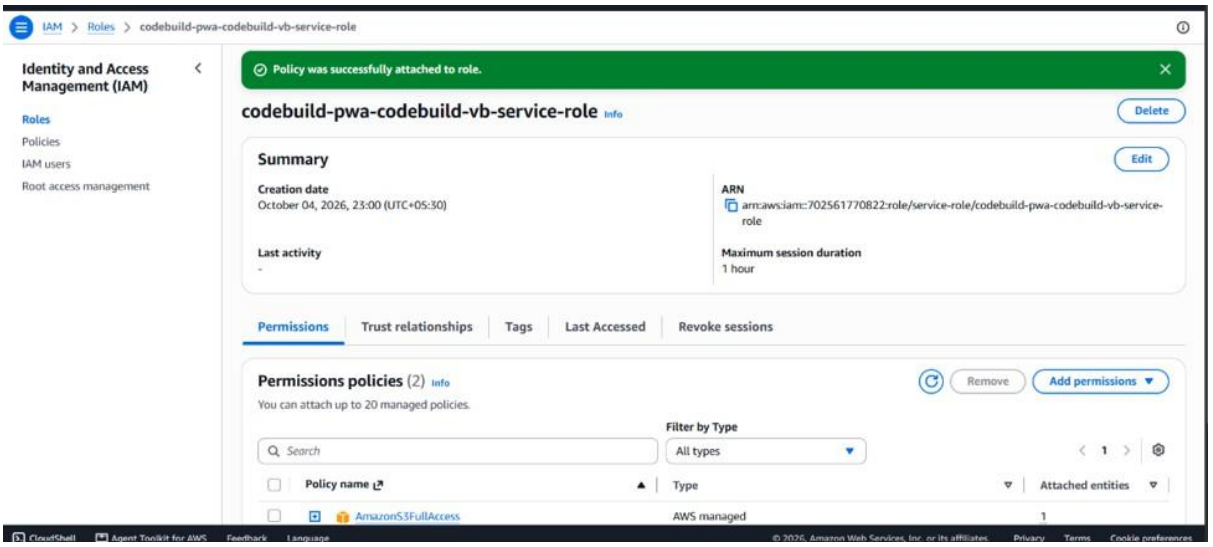
1. Upon pipeline creation, the initial automatic run fails during the **Build** stage due to

missing S3 access permissions for the CodeBuild service role.



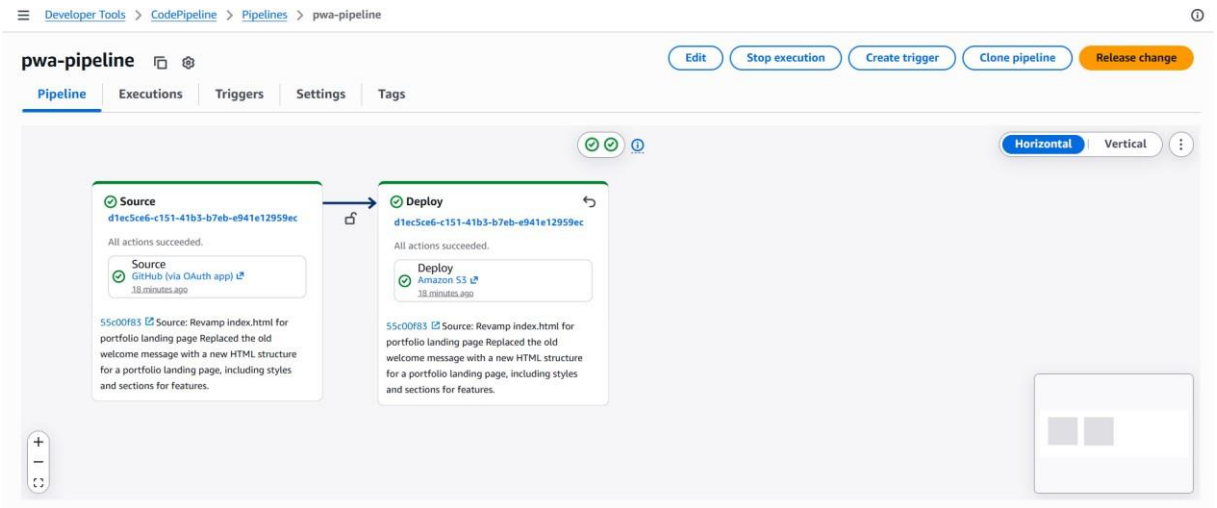
Step 7: Modify IAM Role Permissions

1. Navigate to **CodeBuild > Build projects > pwa-codebuild-vb**.
2. Go to **Build details**, locate the **Service role** ARN, and click it to open the AWS IAM Console in a new tab.
3. In the IAM Role (codebuild-pwa-codebuild-vb-service-role), click **Add permissions > Attach policies**.
4. Search for AmazonS3FullAccess, select it, and click **Add permissions**.



Step 8: Retry Pipeline Execution

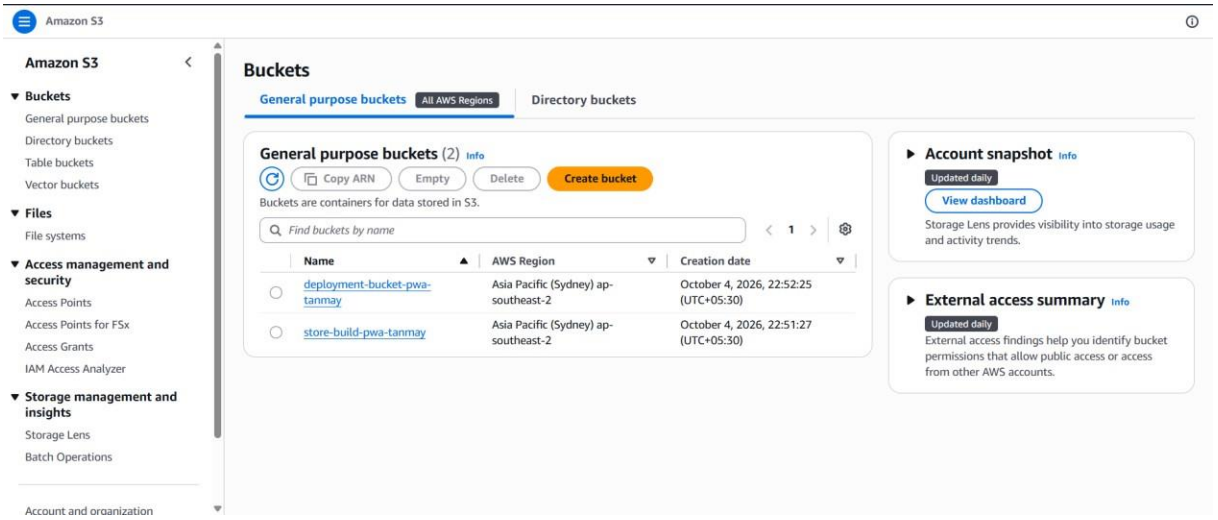
1. Return to **CodePipeline > Pipelines > pwa-pipeline**.
2. Verify that all two stages (**Source** and **Deploy**) [Build removed due to continuous error] complete successfully with green status indicators.

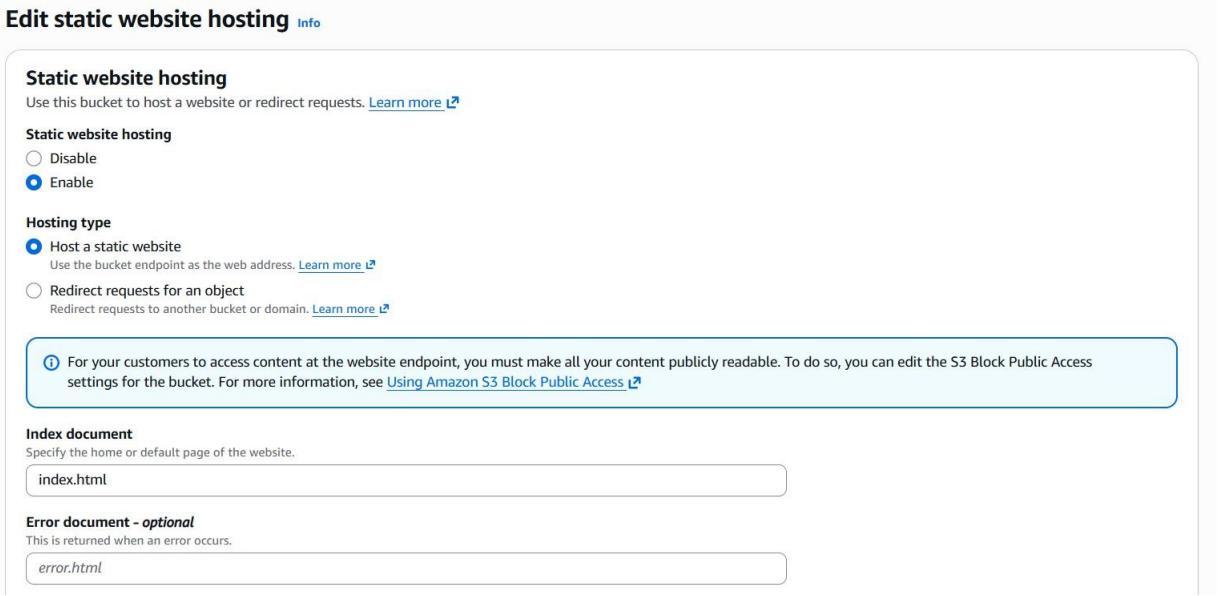


Phase 4: S3 Static Website Hosting & Public Access Policy

Step 9: Enable Static Website Hosting

1. Go to **Amazon S3 > Buckets > deployment-bucket-pwa-vb**.
2. Select the **Properties** tab and scroll to **Static website hosting**.
3. Click **Edit**, select **Enable**, choose **Host a static website**, and set **Index document** to index.html. Save changes.





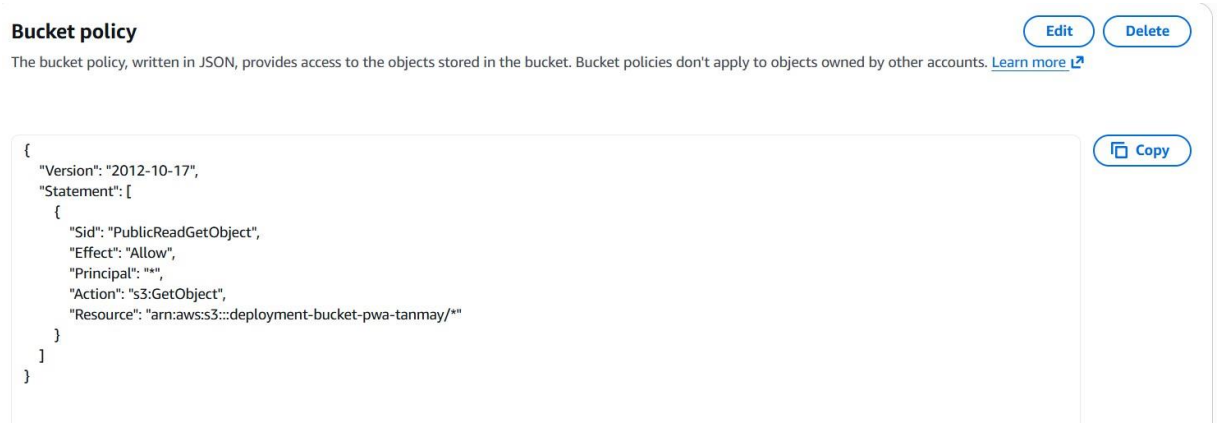
Step 10: Configure Public Bucket Policy

1. Select the **Permissions** tab of deployment-bucket-pwa-vb.
2. Disable **Block all public access** if enabled.
3. Scroll down to **Bucket Policy**, click **Edit**, and paste the following JSON policy:

JSON

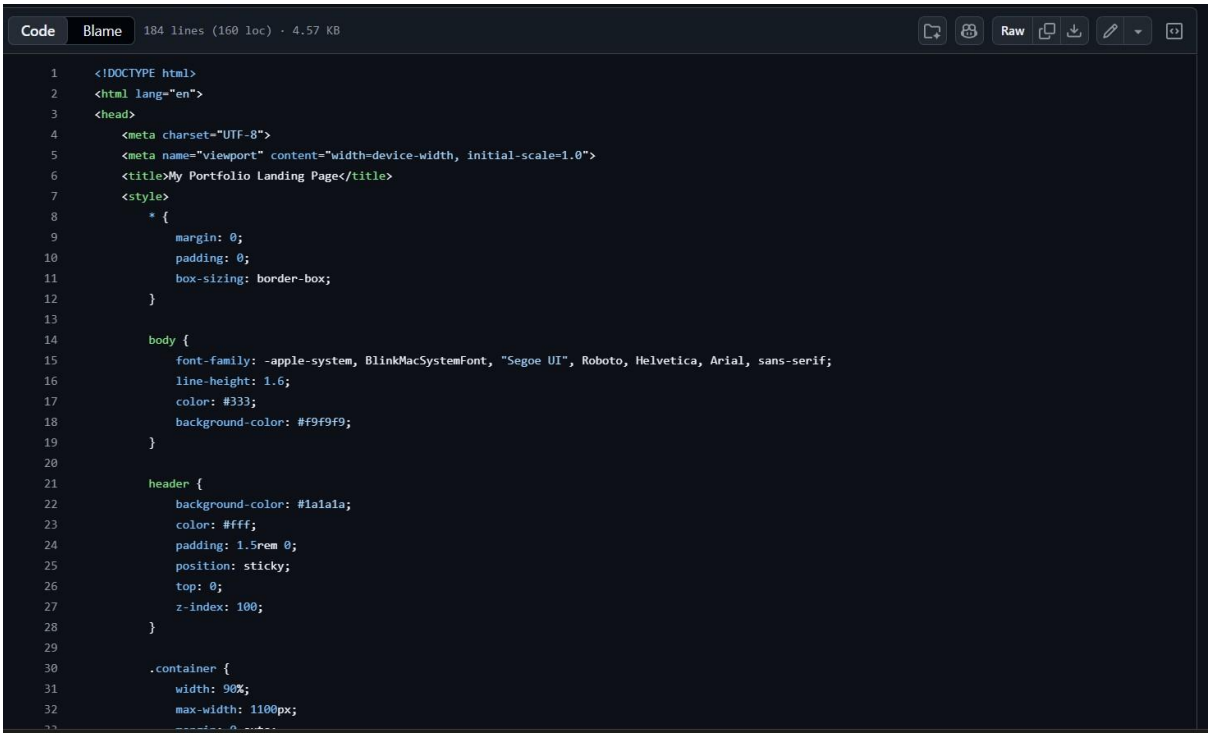
```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "PublicReadGetObject",
   "Effect": "Allow",
   "Principal": "*",
   "Action": [
   "s3:GetObject"
   ],
   "Resource": [
    "arn:aws:s3:::deployment-bucket-pwa-vb/*"
   ]
```

```
    }
  ]
}
```



Step 11: Verify Initial Live Website

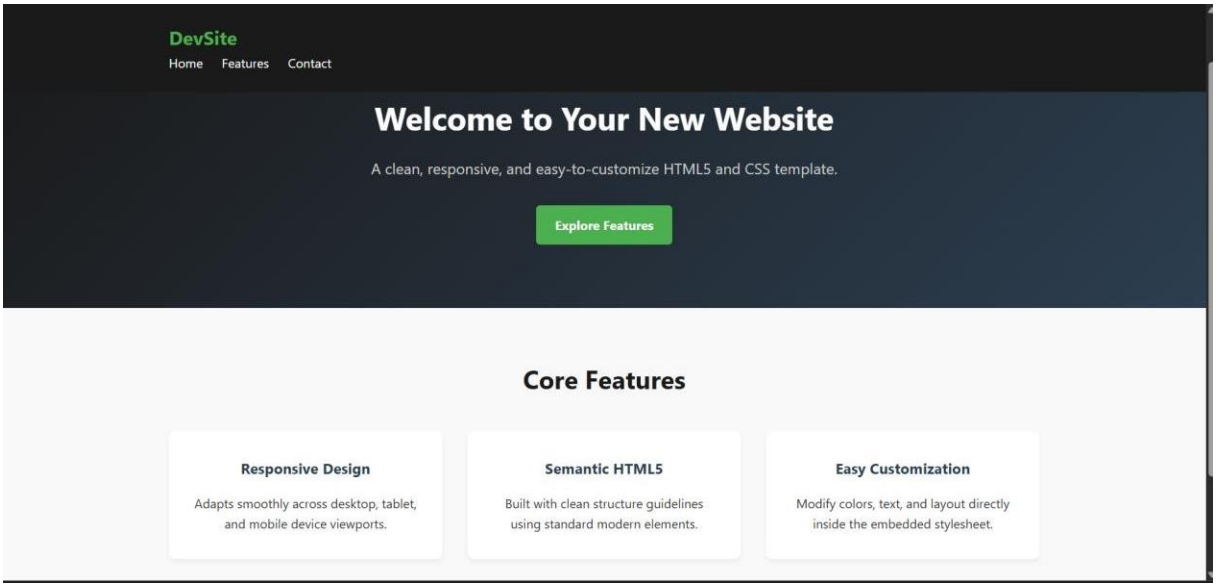
1. Navigate back to **Properties > Static website hosting** on the S3 bucket.
2. Click the generated **Bucket website endpoint** URL (e.g., http://deployment-bucket-pwa-vb.s3-website-us-east-1.amazonaws.com).
3. Verify that the web application renders correctly in the browser.



MyFirstRepo / index.html ↑ Top

Code Blame 184 lines (160 loc) · 4.57 KB Raw

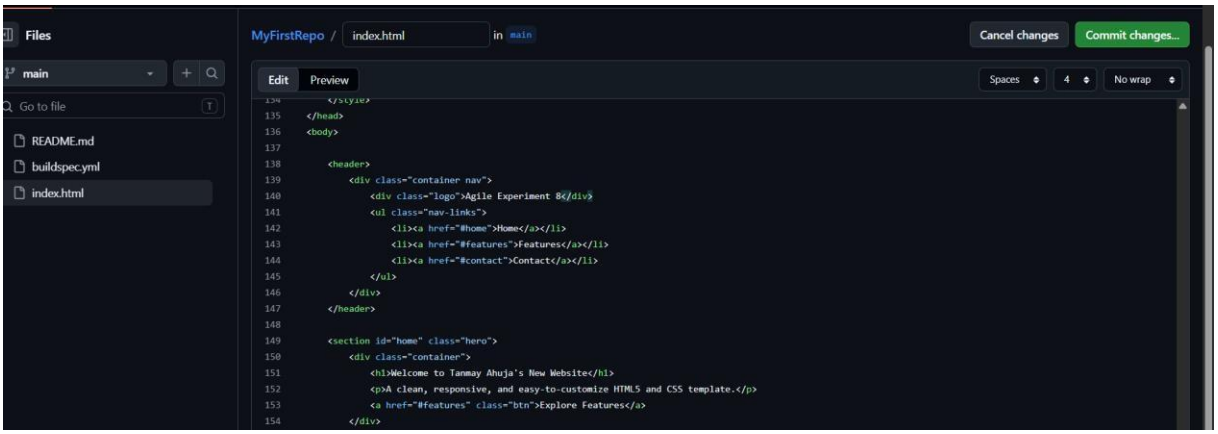
```
156
157     <section id="features">
158         <div class="container">
159             <h2 class="section-title">Core Features</h2>
160             <div class="features-grid">
161                 <div class="card">
162                     <h3>Responsive Design</h3>
163                     <p>Adapts smoothly across desktop, tablet, and mobile device viewports.</p>
164                 </div>
165                 <div class="card">
166                     <h3>Semantic HTML5</h3>
167                     <p>Built with clean structure guidelines using standard modern elements.</p>
168                 </div>
169                 <div class="card">
170                     <h3>Easy Customization</h3>
171                     <p>Modify colors, text, and layout directly inside the embedded stylesheet.</p>
172                 </div>
173             </div>
174         </div>
175     </section>
176
177     <footer id="contact">
178         <div class="container">
179             <p>&copy; 2026 DevSite. All rights reserved.</p>
180         </div>
181     </footer>
182
183 </body>
184 </html>
```

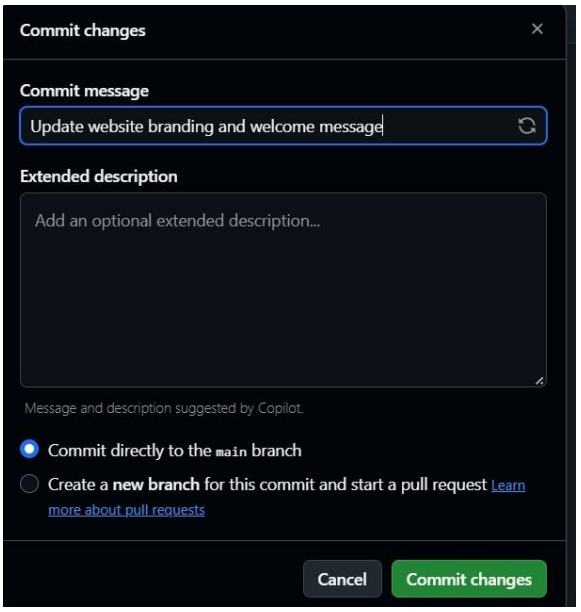


Phase 5: Testing Continuous Integration & Automated Redeployment

Step 12: Push Code Update to GitHub

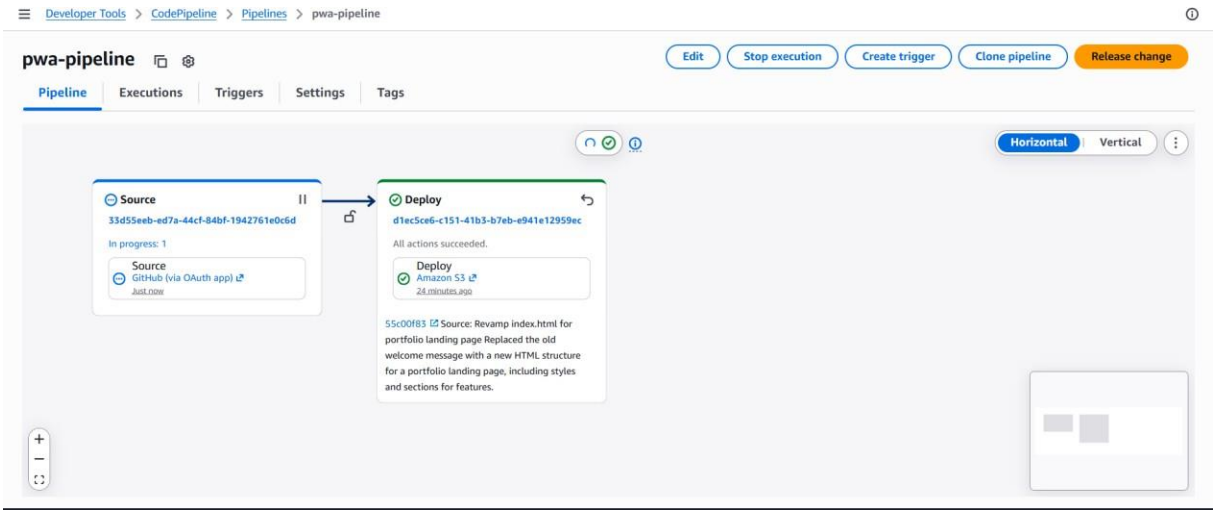
1. Open the GitHub repository (vishal003/Car-pwa-deploy-on-aws) and edit index.html.
2. Modify banner text (e.g., update description text to include *"A.P. Shah Institute of Technology"*).
3. Commit and push changes directly to the master branch.

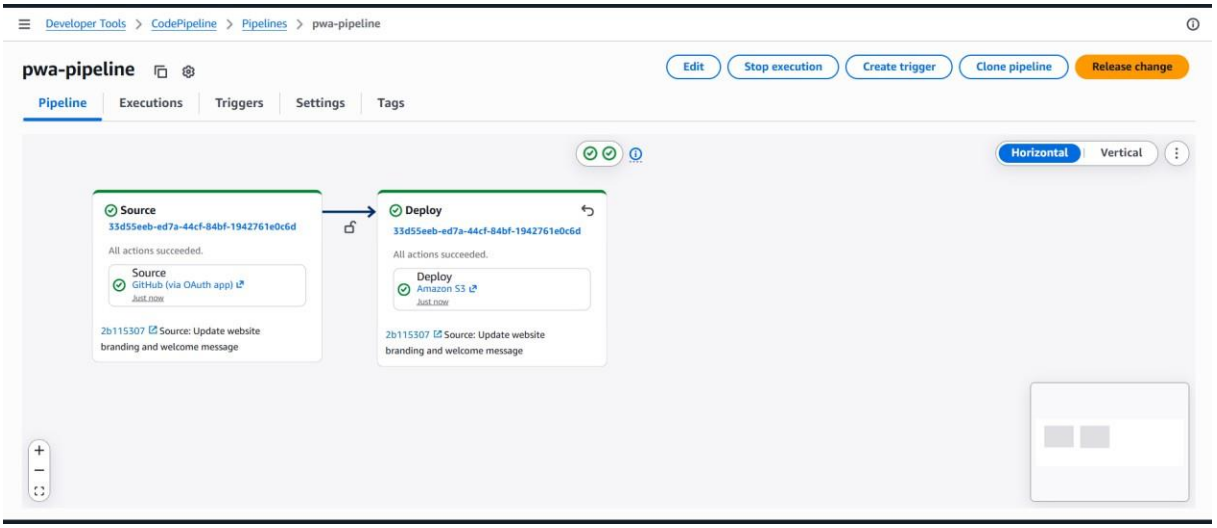




Step 13: Verify Auto-Trigger and Live Web Page Update

1. Open **AWS CodePipeline**. Observe that the pipeline automatically detects the commit (Update banner text...) and initiates execution (In progress).
2. Once the pipeline completes successfully, refresh the live S3 website endpoint URL.
3. Confirm that the webpage updates in real time with the new changes without any manual deployment steps.





Step 14: Resource Cleanup

1. Delete the created AWS CodePipeline (pwa-pipeline).
2. Delete the AWS CodeBuild project (pwa-codebuild-vb).
3. Empty and delete created Amazon S3 buckets (store-build-artifact-vb and deployment-bucket-pwa-vb).
4. Delete associated IAM Roles.

5. Conclusion

In this experiment, an automated end-to-end continuous integration and continuous deployment (CI/CD) pipeline was successfully established using AWS Developer Tools. By integrating GitHub with AWS CodePipeline, AWS CodeBuild, and Amazon S3, source code revisions were automatically fetched, compiled, and deployed without manual intervention. Granting appropriate IAM permissions ensured seamless access across services, while S3 static website hosting enabled real-time web delivery of application updates. Overall, this experiment demonstrated how cloud-native automation accelerates software release cycles, minimizes operational errors, and ensures high availability in modern software development workflows.